

REINFORCE/A2C 기반 따릉이 재배치 실험 보고서

73 일 시간순 평가 프로토콜에서 본 A2C 의 안정성과 REINFORCE 의 seed 민감도

작성자: 박제영(A73024)

Abstract

본 문서는 REINFORCE with Value Baseline 과 A2C 를 중심으로 서울 25 개 구 따릉이 재배치 실험을 정리한다. 연구 질문은 수요예측과 Top-K 후보 행동 구조를 적용한 재배치 환경에서 TD 기반 A2C 가 Monte Carlo 기반 REINFORCE 보다 더 안정적인가이다. 평가는 2025-10-20 부터 2025-12-31 까지 73 일 holdout 으로 수행했고, MostImbalanced baseline 대비 Delta 를 주 지표로 사용했다. MostImbalanced 는 현재 트럭 적재 상태에 따라 자전거가 가장 과잉이거나 부족한 정류소를 선택하는 학습 없는 규칙 기반 기준 정책이다. Top-K12 전체 실험에서 A2C 는 Best Delta 평균 +13.0, baseline 초과 17/25 구였고, REINFORCE 는 -8.4, 8/25 구였다. Seed 반복 실험에서도 A2C 의 Best seed std 중앙값은 1.0 으로 REINFORCE 24.4 보다 낮았다.

1. 문제 정의

연구 질문: 수요예측 feature 와 Top-K 후보 행동 구조를 적용한 따릉이 재배치 환경에서 TD 기반 A2C 는 Monte Carlo 기반 REINFORCE 보다 더 안정적인가? 이 질문을 확인하기 위해 평균 성능뿐 아니라 학습곡선, Top-K ablation, seed 반복 실험, 구별 Best/Worst 결과를 함께 비교했다.

따릉이 재배치는 정류소별 자전거 부족과 거치 공간 부족을 줄이기 위해 재배치 트럭의 다음 방문 정류소를 순차적으로 선택하는 문제다. Reward 는 stockout, full, 이동 비용을 음수로 계산하므로 0 에 가까울수록 좋고, Delta 가 양수이면 MostImbalanced baseline 보다 좋은 성능이다.

```
maximize episode reward = minimize(stockout penalty + full penalty + travel cost)
```

2. State, Action, Reward

State 에는 현재 재고, capacity, 트럭 상태, 시간 정보, 1 시간 수요예측 feature 를 포함했다. Action 은 전체 정류소 직접 선택이 아니라 매 step 마다 수요예측 기반 Top-K 후보 중 하나를 선택하는 방식이다.

obs_dim 은 구별 정류소 수와 보조 feature 사용 여부에 따라 달라진다. 예를 들어 forecast feature + Top-K 12 기준으로 강남구는 obs_dim=1126, 강동구는 obs_dim=821, 관악구는 obs_dim=611 이었다. 따라서 policy/value network 는 각 구별 환경 생성 후 env.observation_space.shape[0]에서 입력 차원을 읽어 초기화했다. 행동 차원은 Top-K 후보 수와 같으므로 Top-K 12 는 n_actions=12, Top-K 9 는 n_actions=9 이다.

```
pred_net_1h = pred_returns_1h - pred_rentals_1h
projected_bikes = current_bikes + pred_net_1h
Delta = model_eval_reward - MostImbalanced_eval_reward
```

수요예측 feature 를 넣은 이유는 현재 재고만으로는 가까운 미래의 stockout/full 위험을 알기 어렵기 때문이다. 현재는 균형 상태처럼 보여도 다음 1 시간 동안 대여가 몰리면 자전거 부족이 생기고, 반납이 몰리면 거치 공간 부족이 생긴다. 따라서 과거 학습 구간으로 만든 구별 1 시간 수요예측 parquet 을 episode 의 정류소·시간대와 join 해 observation 뒤에 붙였다. 이는 reward 를 바꾸는 것이 아니라 같은 reward 를 더 잘 얻기 위한 state 보강이다.

Feature	의미	기대 역할
pred_rentals_1h	1 시간 예상 대여량	부족해질 정류소 탐지
pred_returns_1h	1 시간 예상 반납량	거치 공간 부족 탐지
pred_net_1h	예상 반납 - 예상 대여	순수요 방향 확인
projected_deviation	예측 후 목표 재고 편차	재배치 우선순위 판단

Top-K 후보 행동 구조는 전체 정류소를 직접 선택하는 큰 action space 를 줄이기 위해 사용했다. 구마다 정류소 수가 수십 개에서 200 개 이상까지 달라지므로, 모든 정류소를 action 으로 두면 대부분의 학습 시도가 현재 상황과 무관한 선택에 쓰

인다. 먼저 수요예측 불균형, 이동거리 penalty, 권역 penalty 로 후보 점수를 만들고, 점수가 높은 K 개 정류소만 남긴 뒤 agent 는 그중 rank 하나를 선택한다.

단계	구현 내용
1	현재 재고, 목표 재고, 1 시간 예측 수요로 예상 불균형 계산
2	이동거리와 권역 penalty 를 반영해 candidate score 계산
3	score 가 높은 정류소를 Top-K 후보로 선택
4	action space 를 전체 정류소가 아니라 0 ... K-1 후보 rank 로 제한
5	선택된 rank 를 실제 정류소 id 로 변환해 환경 step 에 전달

candidate_score = forecast_imbalance - travel_coef * travel_distance - zone_penalty
action = categorical_policy(state) # action is candidate rank 0 ... K-1
station_id = topk_candidates[action]

최종 실험 runner 의 후보 점수 기본값은 travel_coef=0.20, zone_mode=static3, zone_penalty=1.0 이다. static3 는 정류소를 3 개 권역으로 나누고 현재 트럭 권역과 다른 후보에 작은 penalty 를 주어 과도한 장거리 이동을 줄이는 설정이다.

평가 reward 는 원본 환경의 reward 를 사용했고, 본 실험의 계수는 다음과 같다.

r_t = w_stockout*stockout_t + w_full*full_t + w_travel_km*travel_km_t + w_travel_step*travel_step_t		
항목	값	의미
w_stockout	-1.0	대여 수요를 만족하지 못한 자전거 수 penalty
w_full	-0.8	반납 수요를 수용하지 못한 자전거 수 penalty
w_travel_km	-0.008	트럭 이동거리 penalty
w_travel_step	-0.002	트럭 이동 중 10 분 step penalty
urgent_bonus	0.0	평가에서는 추가 bonus 사용 안 함
shaping_scale	0.0	평가에서는 reward shaping 사용 안 함

3. 알고리즘

REINFORCE 는 episode 가 끝난 뒤 reward-to-go 를 계산하는 Monte Carlo policy gradient 다. A2C 는 Actor 와 Critic 을 함께 학습하고 1-step TD target 으로 advantage 를 계산한다. 이번 실험에서는 A2C 가 더 낮은 분산과 더 빠른 TD feedback 덕분에 안정적이었다.

3.1 네트워크 모델과 optimizer

항목	구성	이유
Policy/Actor	obs_dim -> 256 -> 256 -> n_actions	구별 obs_dim 이 달라져도 같은 MLP 구조로 비교
Value/Critic	obs_dim -> 256 -> 256 -> 1	REINFORCE baseline 과 A2C critic 을 같은 용량으로 맞춤
Optimizer	REINFORCE: policy 3e-4/value 1e-3, A2C: actor 1e-4/critic 3e-4	policy 변화는 완만하게, value/critic 추정은 빠르게 분리
Gamma	0.99	하루 episode 안에서 늦게 나타나는 재고 영향을 반영
Advantage normalization	사용	구별 reward scale 차이와 policy gradient scale 완화
Update	REINFORCE: episode 종료 후 1 회 / A2C: batch transition	MC return 과 1-step TD feedback 차이를 명확히 비교
Entropy	사용 안 함	추가 regularizer 없이 기본 policy/value update 효과 확인
Gradient clipping	사용 안 함	최종 REINFORCE/A2C 실험 기준
Mask	invalid action logit = -1e9	Top-K 후보 밖 또는 선택 불가능 action 제거

3.2 Loss 함수(Python code)

아래 코드는 실제 agent 코드의 핵심 loss 계산 흐름을 보고서용으로 줄인 것이다. REINFORCE 는 return 전체를 이용하고, A2C 는 1-step TD target 으로 advantage 를 계산한다.

REINFORCE: returns = discounted_reward_to_go(rewards, gamma) advantage = returns - V(s) policy_loss = -log pi(a s) * advantage A2C: target = r + gamma * V(s') advantage = target - V(s) actor_loss = -log pi(a s) * advantage
Actual REINFORCE loss code: values = value(states) advantages = returns_t - values.detach() policy_loss = mean([-logp * advantages[i] for i, logp in enumerate(traj.logp)]) value_loss = F.mse_loss(values, returns_t)

```
Actual A2C loss code:
value_target = reward + gamma * (1 - done) * value(next_state)
advantage = value_target - value(state)
actor_loss = -(log_prob * advantage).mean()
critic_loss = F.mse_loss(value(state), value_target)
```

A2C 의 actor_loss 와 critic_loss 는 하나의 total_loss 로 합치지 않았다. actor_loss 는 actor optimizer 로, critic_loss 는 critic optimizer 로 각각 분리해 업데이트했으므로 critic loss weight 를 별도로 두지 않았다.

4. 실험 요약

4.1 학습/평가 데이터

구분	기간	일수	용도
Train	2025-01-01 ~ 2025-10-19	292 일	policy/value 학습
Eval holdout	2025-10-20 ~ 2025-12-31	73 일	중간 평가와 최종 비교

한 episode 는 하루 단위 환경이다. 학습 중 50 episode 마다 73 일 holdout 전체 평균 reward 를 계산했다. 따라서 history 의 평가점 11 개는 평가 날짜 수가 아니라 checkpoint 수를 의미한다.

4.2 MostImbalanced baseline

상태	선택 기준	의미
트럭이 비어 있음	bikes - target 이 가장 큰 정류소	잉여 자전거를 실기 위함
트럭이 가득 참	target - bikes 가 가장 큰 정류소	부족 정류소에 내려놓기 위함
부분 적재	abs(bikes - target)이 가장 큰 정류소	가장 불균형한 정류소 처리

MostImbalanced 는 학습하지 않는 규칙 기반 정책이며, 모든 결과의 기준점으로 사용했다. Delta 는 모델 reward 에서 MostImbalanced reward 를 뺀 값이고, 양수이면 baseline 보다 좋다.

4.3 실험 시나리오

단계	실험	목적
1	Full baseline run	25 개 구, Top-K 12, seed 42 로 기본 성능 확인
2	Best/Worst subset	잘 되는 구와 어려운 구를 뽑아 후속 실험 대상으로 선정
3	Top-K ablation	K=3/6/9/12/15 비교, subset screening 이므로 selection bias 가능성 있음
4	Confirmation	선택 K 에서 500 episode 재학습
5	Seed validation	seed 42/123/777 반복, 구별 seed 표준편차 계산
6	Final full run	선택 설정으로 서울 25 개 구 전체 학습

실험은 세 단계로 진행했다. 먼저 서울 25 개 구 전체를 Top-K 12 기준으로 학습해 기본 성능을 확인했다. 그 다음 Best/Worst 구를 대상으로 Top-K 후보 수를 3, 6, 9, 12, 15 로 바꾸어 screening 을 수행했다. 마지막으로 선택한 후보 설정 에서 seed 42, 123, 777 을 반복해 결과의 재현성과 분산을 확인했다. 학습 중 rollback 은 사용하지 않았고, 중간 평가 중 가장 좋은 모델을 Best checkpoint 로만 저장했다.

항목			설정			
대상 지역			서울 25 개 구			
평가 기간			2025-10-20 ~ 2025-12-31, 73 일			
학습 길이			500 episodes			
평가 주기			50 episodes			
주 지표			Delta = model reward - MostImbalanced reward			
Seed 검증			42, 123, 777			
Top-K 비교			3, 6, 9, 12, 15			
실험	알고리즘	구	Best Δ 평균	Best 승리	Final Δ 평균	Final 승리
Full TopK12	A2C	25	+13.0	17	+3.2	14
Full TopK12	REINFORCE	25	-8.4	8	-35.5	6
Final TopK9	A2C	25	+11.6	16	-0.4	12
Final TopK9	REINFORCE	25	+0.2	13	-24.0	9
VAE TopK9 BW	A2C+VAE	10	-3.1	4	-15.5	4
VAE TopK9 BW	REINFORCE+VAE	10	-13.1	4	-30.2	4
Bandit TopK9 BW	Contextual Bandit	10	-31.4	0	-50.2	0
Bandit TopK12 all	Contextual Bandit	25	-260.4	0	-283.6	0

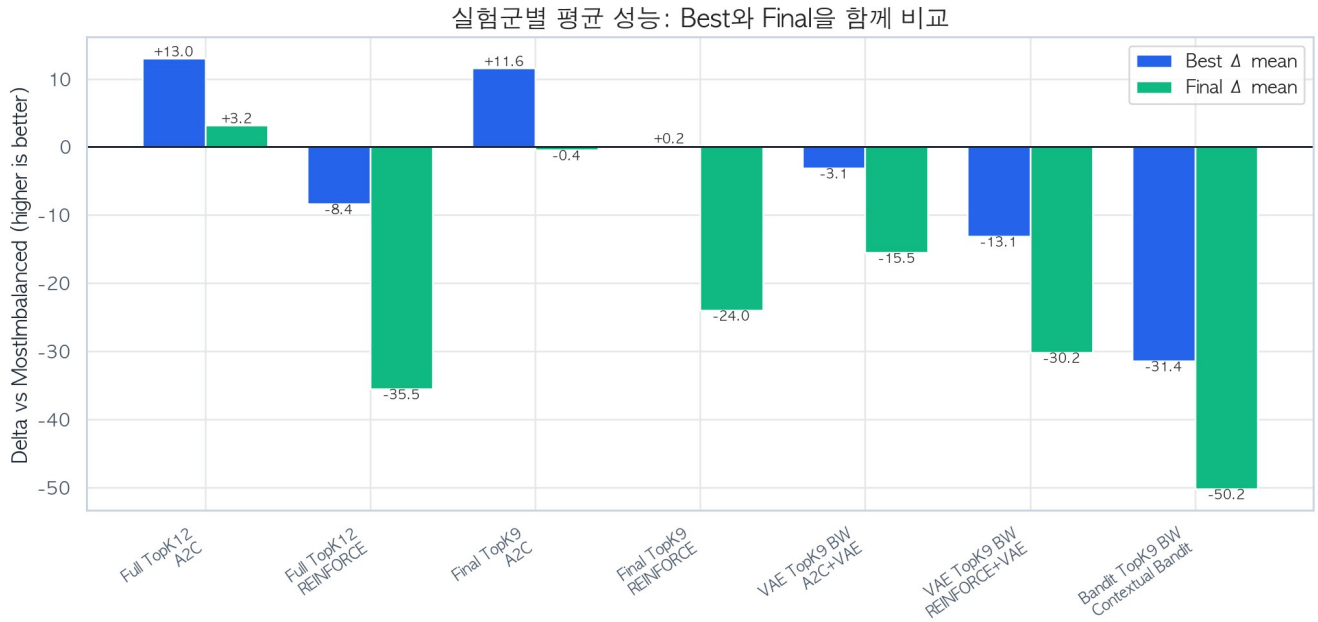


Figure 1. 실험군별 Best/Final 평균 Delta

전체 요약에서 가장 중요한 관찰은 A2C 가 Best 와 Final 모두에서 REINFORCE 보다 안정적이라는 점이다. Top-K 12 전체 실험에서 A2C 는 평균 Best Delta 와 Final Delta 가 모두 양수였고, REINFORCE 는 Best 가 낮거나 Final 에서 하락하는 경향이 더 강했다.

REINFORCE 가 Top-K 후보 수에 더 민감했던 이유는 Monte Carlo policy gradient 특성과 연결된다. REINFORCE 는 episode 전체 reward-to-go 로 업데이트하므로 분산이 크고, 후보 행동 수가 줄면 잘못된 후보를 탐색할 가능성이 줄어 Best 성능은 좋아질 수 있다. 하지만 후보가 너무 좁거나 seed 가 달라지면 특정 행동에 policy 가 빨리 몰려 Final 성능이 흔들릴 수 있다. A2C 는 1-step TD advantage 를 batch transition 단위로 갱신하므로 같은 Top-K 변화에서도 상대적으로 완만하게 반응했다.

5. 구별 결과

구	Baseline	REINFORCE Best Δ	REINFORCE Final Δ	A2C Best Δ	A2C Final Δ	승자
강남구	-350.4	+31.3	+31.3	+31.3	+31.3	Tie
강동구	-334.3	+6.1	+6.1	+6.1	+6.1	Tie
강북구	-13.6	-0.6	-0.8	-0.6	-2.2	REINFORCE(Final)
강서구	-2676.1	+38.7	+38.7	+38.7	+38.7	Tie
관악구	-73.9	-27.7	-28.6	-26.5	-26.5	A2C
광진구	-591.6	+19.7	+19.7	+19.7	+19.7	Tie
구로구	-609.6	+13.2	-114.4	+27.3	+27.3	A2C
금천구	-256.2	+7.1	+7.1	+7.1	+7.1	Tie
노원구	-457.2	+26.7	+26.7	+52.9	-11.3	A2C
도봉구	-82.2	-5.5	-5.5	-5.5	-5.5	Tie
동대문구	-321.1	-10.5	-19.1	+6.3	+6.3	A2C
동작구	-97.8	-5.2	-5.2	-5.2	-17.5	REINFORCE(Final)
마포구	-490.4	+44.6	+44.1	+44.6	+44.6	A2C(Final)
서대문구	-139.3	-17.3	-64.7	-17.3	-31.9	A2C(Final)
서초구	-173.7	+26.5	+26.5	+27.1	-22.2	A2C
성동구	-594.9	+16.7	-46.8	+16.7	-7.5	A2C(Final)
성북구	-102.1	-15.0	-34.3	-15.0	-15.0	A2C(Final)
송파구	-1225.8	-63.5	-155.1	+52.0	-76.4	A2C
양천구	-1083.3	+12.8	-59.3	+52.8	+52.8	A2C
영등포구	-1897.0	-24.2	-24.2	-24.1	-29.7	A2C
용산구	-66.9	-3.8	-3.8	-3.8	-3.8	Tie
은평구	-150.2	-47.2	-112.7	-9.1	-9.4	A2C
종로구	-268.3	+4.7	+4.7	+4.7	+4.7	Tie
중구	-182.6	-29.5	-78.3	+3.3	+3.3	A2C
종량구	-161.6	+7.0	-53.4	+7.0	+7.0	A2C(Final)

승자 판정은 Best Delta 를 우선 기준으로 한다. 표기상 소수 1 자리에서 Best 가 동률이면 Final Delta 가 더 높은 알고리즘을 A2C(Final) 또는 REINFORCE(Final)로 표시하고, Best/Final 모두 같으면 Tie 로 표시한다.

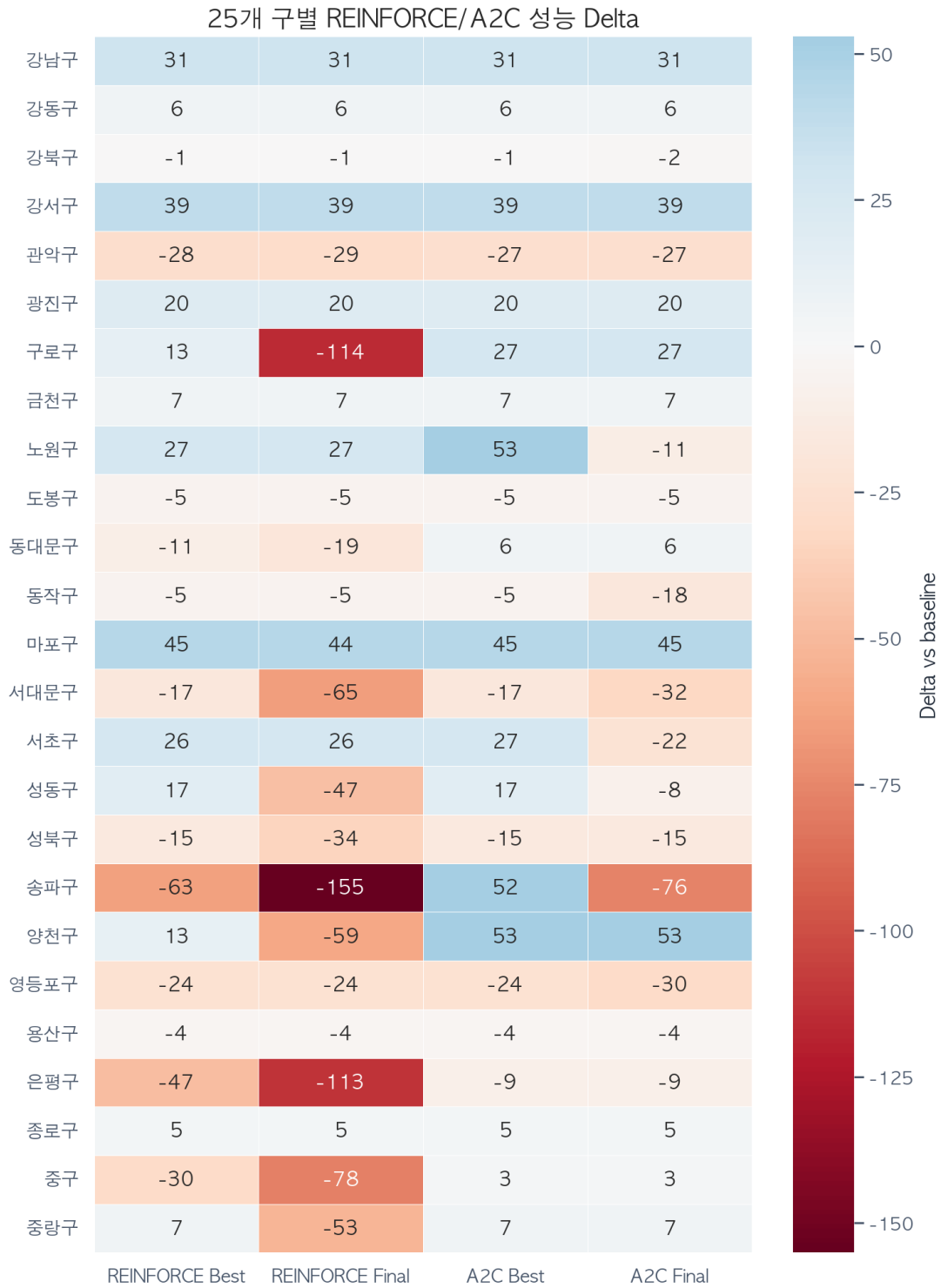
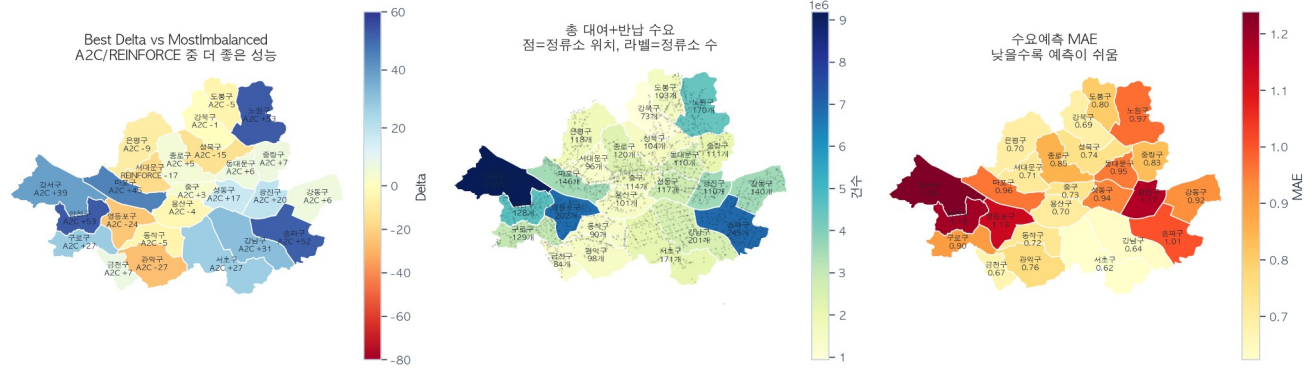


Figure 2. 25개 구별 Delta heatmap

서울 25개 구 진단 지도: 성능, 수요 규모, 예측 난이도



해석: 성능 지도만 단독으로 보지 않고, 정류소/수요 규모와 예측 MAE를 함께 보면 구별 난이도와 학습 결과 차이를 설명하기 쉽다.

Figure 3. 서울 25 개 구 진단 지도

서울 지도는 결과 해석을 공간적으로 보완한다. 왼쪽 지도는 구별 Best Delta와 우수 알고리즘을 보여주고, 가운데 지도는 정류소 위치와 총 수요 규모를 보여주며, 오른쪽 지도는 수요예측 MAE를 보여준다. 이 세 지도를 함께 보면 단순히 어떤 구가 이겼는지뿐 아니라 정류소 밀도, 수요 규모, 예측 난이도가 학습 결과 차이에 어떤 배경으로 작용했는지 설명할 수 있다.

5.1 Best/Worst 구

알고리즘	Best 3	Worst 3
REINFORCE	마포구 +44.6, 강서구 +38.7, 강남구 +31.3	송파구 -63.5, 은평구 -47.2, 중구 -29.5
A2C	노원구 +52.9, 양천구 +52.8, 송파구 +52.0	관악구 -26.5, 영등포구 -24.1, 서대문구 -17.3

Best/Worst 구는 이후 하이퍼파라미터 screening 대상이 되었다. 모든 조합을 25개 구 전체에 적용하면 계산량이 지나치게 커지므로, 먼저 대표적으로 잘 되는 구와 어려운 구에서 후보를 좁힌 뒤 최종 후보를 전체 구로 확장하는 방식을 사용했다.

6. 학습곡선

학습 진행에 따른 평가 성능 변화

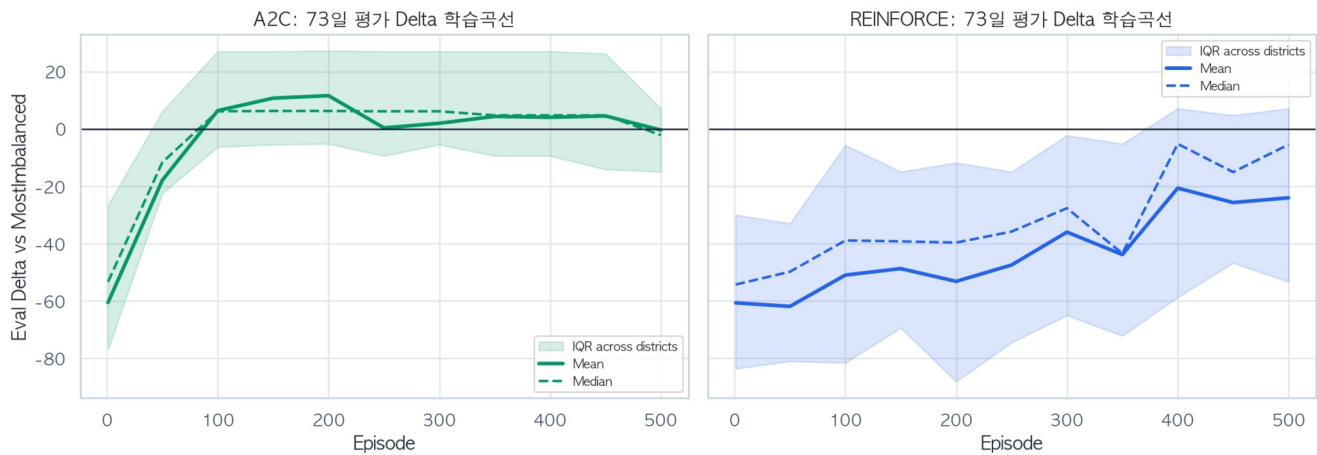


Figure 4. 평가 checkpoint 별 학습곡선

학습곡선은 평가 checkpoint 마다 73일 holdout 평균 Delta를 계산한 것이다. A2C는 초반부터 baseline 근처로 이동하고 비교적 안정적인 반면, REINFORCE는 구별 편차가 크다.

알고리즘	Best-Final gap 평균	중앙값	최대 gap	최대 gap 구	Best ep 평균	Best ep 중앙값	100ep 이내 Best 구 수
REINFORCE	24.3	0.2	127.7	구로구	274.0	300.0	6
A2C	12.0	0.0	128.4	송파구	92.0	100.0	20

Best-Final gap 은 Best Delta 에서 Final Delta 를 뺀 값이다. A2C 의 평균 gap 은 12.0, REINFORCE 는 24.3 로 REINFORCE 가 학습 중 좋았던 정책을 마지막까지 유지하지 못한 경우가 더 컸다. Best episode 중앙값도 A2C 100, REINFORCE 300 로, 현재 로그 기준 A2C 가 더 이른 checkpoint 에서 좋은 정책을 찾는 경향을 보였다.

Best/Worst 구의 실제 학습곡선: aggregate 뒤의 개별 구 반응



Figure 5. Best/Worst 구별 실제 학습곡선

평균 학습곡선만 보면 개별 구의 차이가 가려진다. 위 그림은 알고리즘별 Best 3 구와 Worst 3 구를 따로 그려, 잘 되는 구는 초반부터 baseline 위로 올라가지만 어려운 구는 같은 설정에서도 0 선 아래에 머무를 수 있음을 보여준다.

7. Top-K ablation

알고리즘	Top-K	구	Best 중앙값	Best 승리	Final 중앙값	Final 승리
A2C	3	12	+14.7	7	-5.9	6
A2C	6	12	+13.7	7	-16.1	5
A2C	9	12	+15.3	7	-5.9	6
A2C	12	12	+8.0	7	-16.1	5
A2C	15	12	+8.0	7	-5.9	6
REINFORCE	3	10	+12.5	6	-5.3	4
REINFORCE	6	10	+4.2	6	-20.7	4
REINFORCE	9	10	-23.6	3	-65.2	1
REINFORCE	12	10	-22.7	4	-54.8	2
REINFORCE	15	10	-15.1	4	-47.2	2

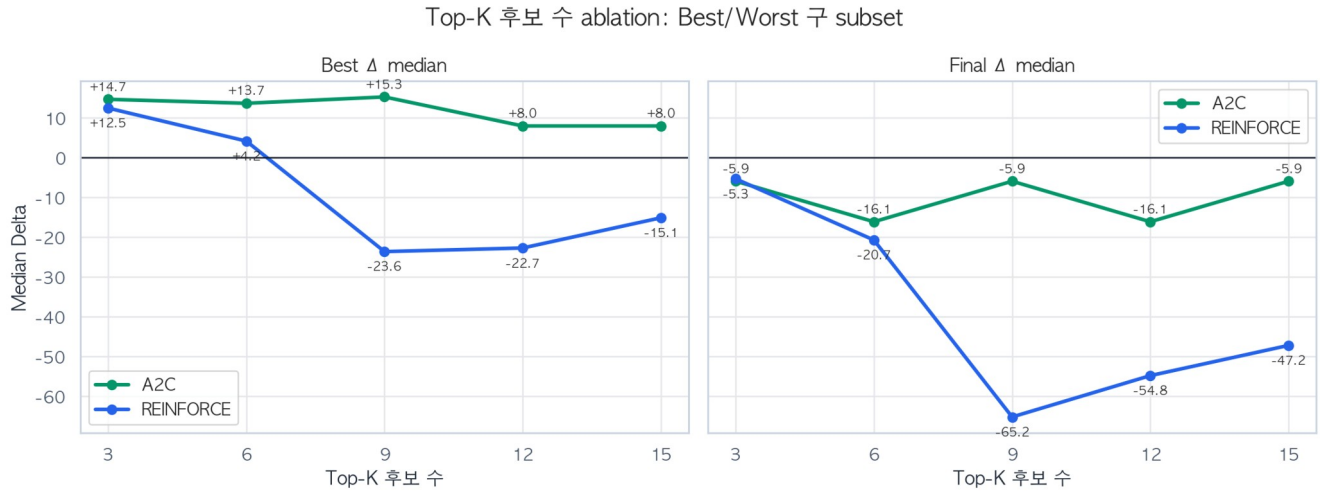


Figure 6. Top-K 후보 수 ablation

Top-K 후보 수는 탐색 난이도와 선택 다양성 사이의 절충점이다. K 가 너무 작으면 좋은 정류소가 후보에서 빠질 수 있고, K 가 너무 크면 policy가 학습해야 할 action 선택지가 늘어난다. 이 ablation은 Best/Worst subset에서 수행했으므로 전체 25개 구 일반화 근거라기보다는 후보 설정을 좁히는 screening으로 해석한다. subset 실험에서는 REINFORCE가 $K=3$ 에서 강했고, A2C는 $K=9\sim 12$ 범위에서 비교적 안정적이었다.

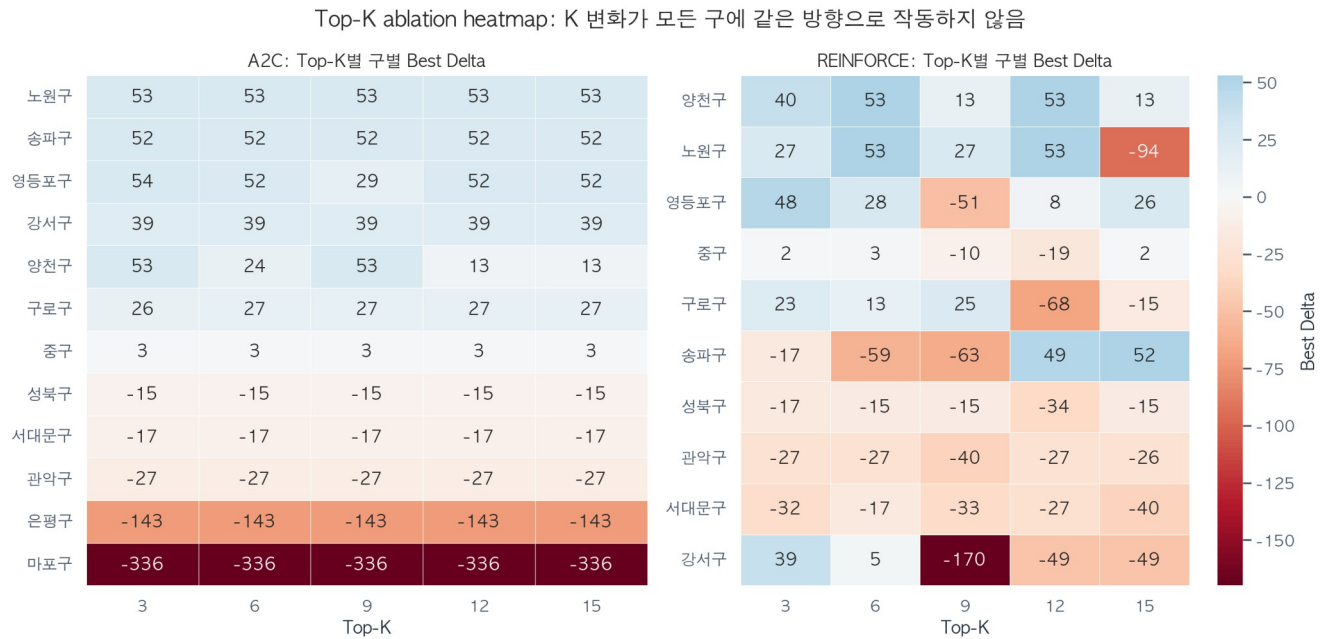


Figure 7. 구별 Top-K 민감도 heatmap

구별 heatmap은 같은 K 값도 구마다 다르게 작동한다는 점을 보여준다. 따라서 최종 K 는 단일 구의 최고값이 아니라 여러 구에서의 중앙값, 승리 수, seed 안정성을 함께 보고 선택했다.

8. Seed 안정성

알고리즘	구 수	Best 평균	Best seed std 평균	Best seed std 중앙값	Final 평균	Final seed std 평균	Final seed std 중앙값
A2C	10	+13.7	10.2	1.0	-3.8	31.4	25.3
REINFORCE	10	-12.3	28.3	24.4	-47.6	49.8	33.6

8.1 Seed 별 요약

알고리즘	Seed	실험 수	Best 평균	Best 표준편차	Best 승리	Final 평균	Final 표준편차	Final 승리
A2C	42	10	+14.4	+33.8	6	-6.9	+38.5	4
A2C	123	10	+21.7	+32.3	7	-9.0	+52.0	4
A2C	777	10	+4.9	+34.7	5	+4.7	+35.1	5
REINFORCE	42	10	-8.6	+30.9	4	-49.3	+59.1	2
REINFORCE	123	10	-10.3	+38.6	2	-48.4	+95.1	2
REINFORCE	777	10	-18.2	+53.7	4	-45.1	+63.7	2

Seed 42/123/777 반복 실험: 구별 seed 표준편차

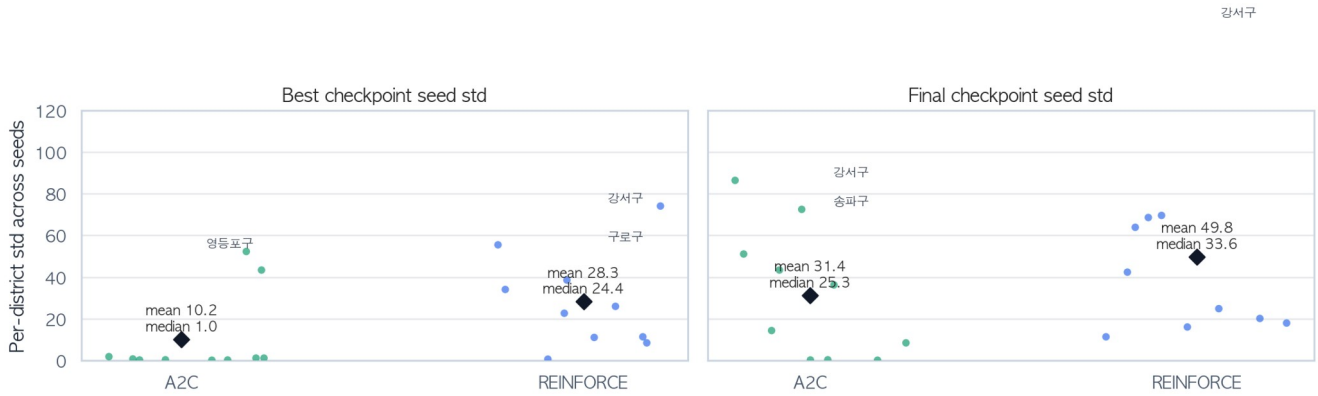


Figure 8. 구별 seed 표준편차 분포

Seed 안정성은 raw 30 회 결과를 한꺼번에 섞어 계산하지 않고, 같은 구 안에서 seed 3 개의 표준편차를 먼저 구한 뒤 알고리즘별로 요약했다. 이 방식은 구별 난이도 차이와 seed 차이를 분리한다. A2C 는 Best seed std 평균 10.2, 중앙값 1.0 으로 대부분의 구에서 안정적이었고 일부 outlier 가 평균을 끌어올렸다. REINFORCE 는 Best seed std 평균 28.3, 중앙값 24.4 로 전반적으로 seed 에 더 민감했다. Final 기준으로는 REINFORCE 강서구가 std 162.9 로 가장 큰 outlier 였고, seed 123 에서 강서구 Final Delta 가 -283.4 까지 하락했다.

A2C 의 Best seed std 평균과 중앙값 차이는 주로 영등포구(Best std 52.3)와 양천구(43.4)가 만든 것이다. Final 기준 outlier 는 강서구(86.4), 송파구(72.5), 구로구(51.1)가 컸다. 따라서 A2C 는 대다수 구에서 안정적이지만 일부 구에서는 후반 checkpoint 안정성이 떨어질 수 있다.

Seed별 Delta 분포: REINFORCE는 분포 폭이 더 넓다

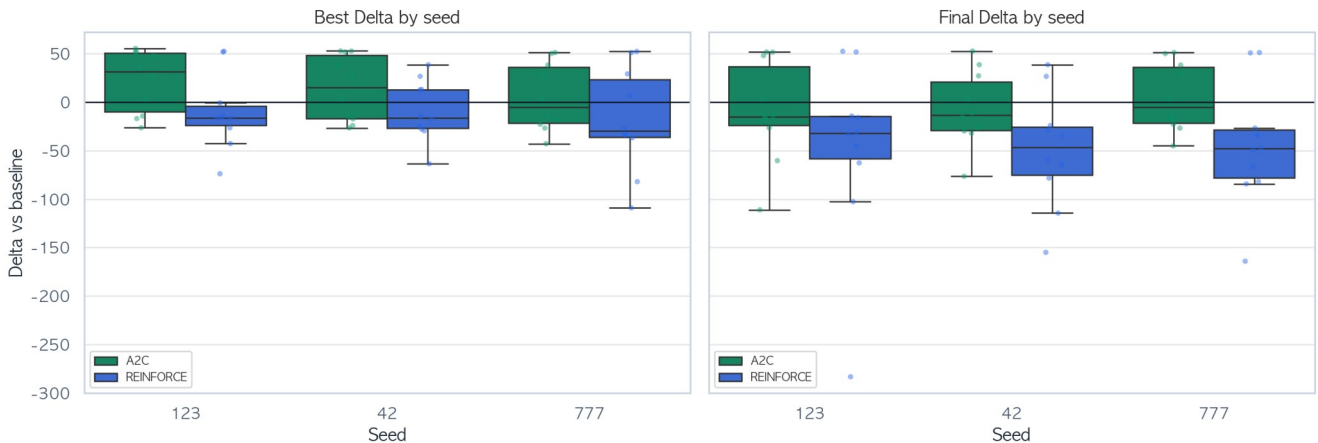


Figure 9. Seed 별 Best/Final Delta 분포

seed 별 raw Delta 분포를 함께 보면 REINFORCE 의 분포 폭이 A2C 보다 넓다. 이는 Monte Carlo reward-to-go 기반 update 가 초기 sampling 과 구별 수요 패턴에 더 민감하다는 해석을 뒷받침한다.

9. VAE 와 Contextual Bandit

알고리즘	구	기존 Best	VAE Best	Best gain 평균	개선 구	기존 Final	VAE Final	Final gain 평균
A2C+VAE	10	+14.4	-3.1	-17.5	1	-6.9	-15.5	-8.6
REINFORCE+VAE	10	-8.6	-13.1	-4.5	6	-49.3	-30.2	19.2

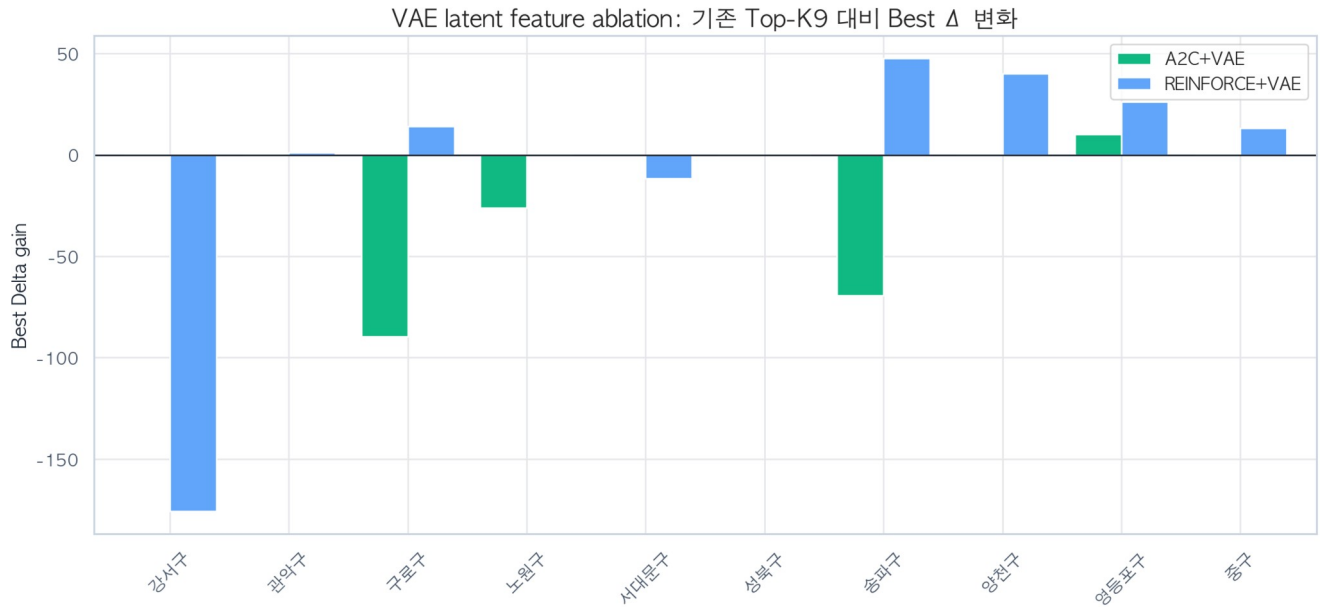


Figure 10. VAE latent 추가에 따른 Best Delta 변화

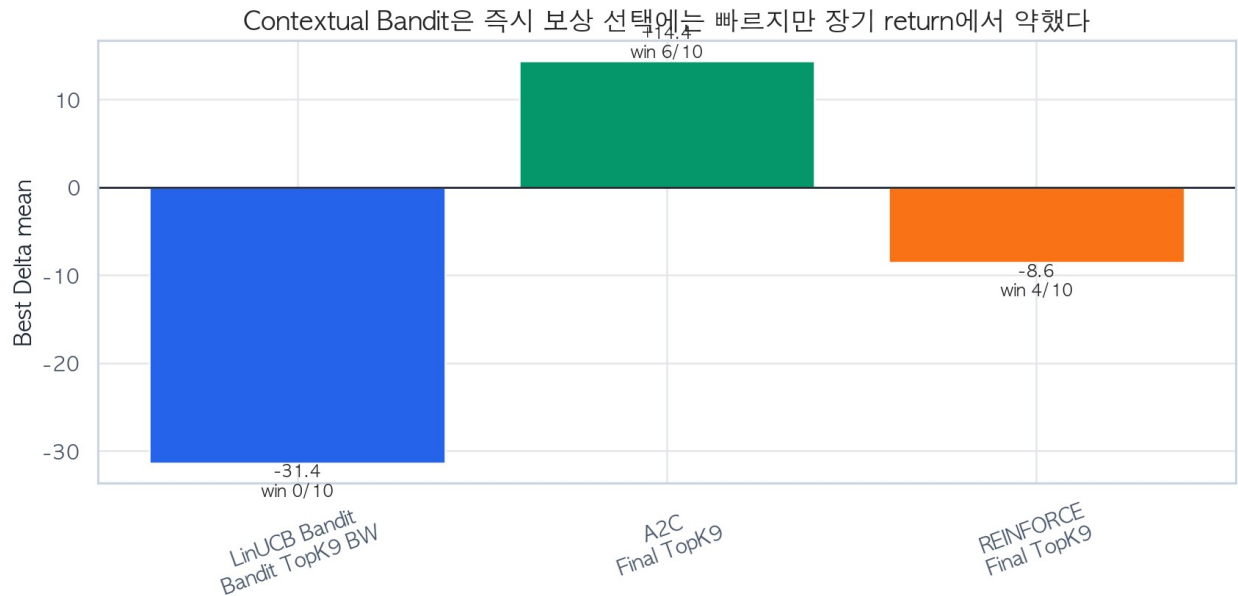


Figure 11. Contextual Bandit 과 RL 비교

VAE 는 정류소별 1 시간 수요 패턴을 작은 latent vector 로 압축해 state 뒤에 추가한 실험이다. VAE gain 은 전체 25 개 구가 아니라 VAE 를 실제로 돌린 동일 10 개 구 subset 에서 seed 42 Top-K9 결과와 비교한 값이다. REINFORCE 일부 구에서는 Final 성능이 개선됐지만 Best 기준으로는 일관되지 않았고, A2C 에서는 critic 이 이미 상태의 장기 가치를 학습하기 때문에 VAE latent 가 noise 처럼 작동한 구가 있었다. Contextual Bandit 은 현재 step 의 context 만 보고 action 을 선택하므로, 현재 이동이 미래 재고에 미치는 장기 효과를 학습하지 못했다.

10. 결론

10.1 그래프 구성의 의미

질문	그림	역할
평균 성능	실험군별 Best/Final 평균 Delta	A2C 와 REINFORCE 의 전체 우열
지역 편차	25 개 구 heatmap	구별로 어려운 지역과 쉬운 지역 확인
공간 해석	서울 25 개 구 진단 지도	성능, 수요 규모, 예측 난이도를 함께 비교
학습 진행	episode 별 평가 학습곡선	학습이 실제로 개선되는지 확인
seed 안정성	구별 seed 표준편차 scatter	REINFORCE 의 variance 와 A2C 안정성 확인
하이퍼파라미터	Top-K ablation	Top-K 선택 근거 제시

최종 결론은 A2C + 수요예측 state + Top-K 후보 action 이 본 담당 범위에서 가장 설득력 있는 모델이라는 점이다. REINFORCE 는 policy gradient 구조를 설명하기 좋은 비교 모델이지만, seed 와 구별 수요 패턴에 민감했다. VAE 와 Contextual Bandit 은 추가 실험으로 의미가 있었지만, 현재 설정에서는 A2C 를 대체할 만큼 일관된 성능 개선을 만들지는 못했다.

Appendix. 산출물 위치

원본 집계 CSV 는 output/results/current_all_experiments_review.csv 에 있고, seed 반복 실험은 output/results/final73_seedci_topk9_summary_detail.csv, Top-K ablation 은 output/results/topk_ablation_73d_summary.csv 에 저장되어 있다. 보고서 그림은 docs/figures 아래에 생성되며, Markdown/Word/PDF 산출물은 각각 docs, output/doc, output/pdf 에 저장된다.